

RTL Coherent NoC based on ARM AMBA 5 CHI Specification

Alireza Monemi

2019

1 Introduction

In this project, an RTL code of a cache coherent network on chip (NoC) is developed. The requirement of this project is to support the minimal ARM AMBA 5 CHI cache coherency protocol for early evaluation of different components that will be developed by other parties in Mont-Blanc 2020 (MB2020) and European Processor Initiative (EPI) projects. The developed RTL code in this project have the following features:

- **Parameterizable:** The developed code is parametrizable in terms of number of buffer size, cache size, number of routers ports, number of endpoints, number of virtual channels (VCs) and etc. This feature allows to test the desired components in different configuration setting.
- **Synthesizable:** This allows to test the design under both simulation (i.e Modelsim, Verilator) and emulation (i.e prototyping on an FPGA platform) environments.
- **Extendable:** Although the AMBA CHI-5 protocol is partially implemented, but the code is developed in such a way that it can be easily modified for supporting additional features in future.
- **Simplified:** Some complex components such as cache and main memory modules are simplified. All cache-based components such as system-cache, request nodes' cache, and snoop filter have two-cycle write and one-cycle read latency, respectively regardless of the cache size. All single/dual port memories have one-cycle read and write latency.

2 Reference NoC

2.1 Reference RTL Code

The NoC RTL code is adapted from ProNoC [?] project. The reference design RTL code has the following features:

Table 1: Reference design NoC specification.

pipeline latency	2-cycle ¹
flow control	wormhole
Number of Virtual channel	Parameterizable
Buffer-width	parameterizable
Topology	Mesh, Torus, Fattree
Routing Protocol	deterministic, partially/fully adaptive

2.2 Reference RTL Code modification

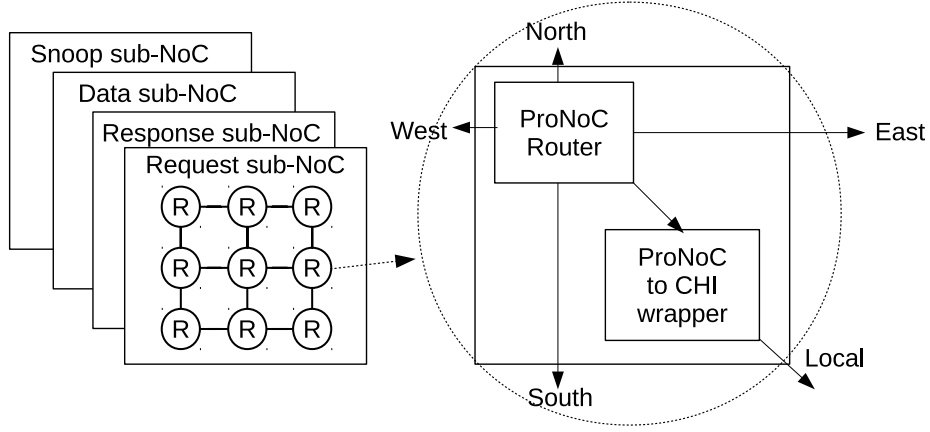


Figure 1: Proposed CHI-based NoC.

Figure 1 illustrates the functional block diagram of the proposed CHI-based NoC. In order to adapt the reference design [?] with AMBA-CHI and MB2020 project specifications the following features have been added to ProNoC:

1. ProNoC originally have a constrain of two-flit minimum packet size. I.e. one header flit for carrying flow-control information and one or more flits for caring data. AMBA CHI specification has single-flit packet size constrain. As a result the flow-control information and data are merged in one flit. To adapted with this configuration, the ProNoC RTL code is modified to support single flit sized packet. Moreover a ProNoC-to-CHI and CHI-TO-ProNoC Wrapper modules are developed to convert AMBA-CHI and ProNoC interfaces signals.
2. In MB2020, some NoC typologies e.g. C-Mesh require that a router supports multiple injection ports. However reference design is not parametrized.

¹The three router processing stages of look-ahead routing, VC allocation and, switch allocation are done in parallel at the first pipeline stage. Flits are passed via the crossbar at the second pipeline stage.

able in terms of injection port numbers. To adapt with this types of typologies this feature has been added to reference router in this project.

3. The AMBA-CHI NoC defines four independent communication channels of Request, Response, Snoop and Data. Each channel is implemented using an isolated physical sub-NoC.

3 Developed CHI-agents

Three types of agent nodes have been developed in this projects: Request node full (RNF), Home node full (HNF), and slave node full (SNF). The agents are development according to AMBA CHI V5 specification. All agents are still under development. AMBA CHI features are gradually added to these nodes during testing evaluation period. This section highlights the agents features.

3.1 Request Node Full

A Request node full (RNF) represents a cache coherent processing element in the system. An RNF is capable of reading transaction commands via trace files and mimics a real processing element. An RNF consists of four sub-agents for handling Request, Response, Data and Snoop channels.

3.1.1 RNF-Request agent

This sub-agents is controlled by the simulator/emulator and propagates appropriate requests to the network. Only a few sub-set of CHI requests are supported by an RNF. The supported requests are chosen according to the supported requests in RTL packet injector which is developed by SemiDynamic. The supported requests are:

- **Readshared:** This request is generated as a response to LOAD-MISS in RNF cache memory. Hence, the initial cache state is invalid (I) state. The RNF expects to receive Data in one of shared (S) or unique (U) states. The final cache state is defined according to the received response from the NoC as shown in Table 3.1.1.

Table 2: ReadShared cache transition state

Initial Cache state	Received response	Final Cache state
I, UCE	CompData_SC	SC
	CompData_UC	UC
	CompData_SD_PD	SD
	CompData_UD_PD	UD

- **ReadUnique:** ReadUnique is a request to STORE-MISS (initial cache state is I). The final cache state will always be dirty (D).

- **CleanUnique:** Request to STORE instruction when the initial cache state is shared clean (SC) or Shared Dirty (SD).
- **WriteBackFull:** Request to write data to the main memory once a cache evict happens.
- **ReadNoSnp:** Read request to a Non-snooperable address region. The received Data from NoC is not cached at the RNF in a system coherent manner. This means that the final cache state will be always I.
- **Evict:** It is a data less transaction and indicates that a Clean cache line is no longer cached by an RN.
- **AtomicLoad, AtomicSWAP, AtomicStore:** All atomic operations except AtomicComp is supported by the NoC.

3.2 Home node full

Home node full (HNF) receives requests from RNFs and handle the necessarily transactions in the NoC. HNF are the most complex part of a cache coherent system and they are the point of coherency and consistency in the system.

The memory address space is divided into several partitions where consecutive cache block addresses go to different home nodes. The destination home node index number is obtained from the following equation:

$$n = \frac{address}{64} \mod (H) \quad (1)$$

where n is the destination HNF index, 64 is the cache block size in bytes, and H is the maximum number of home nodes in the system.

Figure 2 illustrates the functional block diagram of the proposed home-node. HNF consists of following modules :

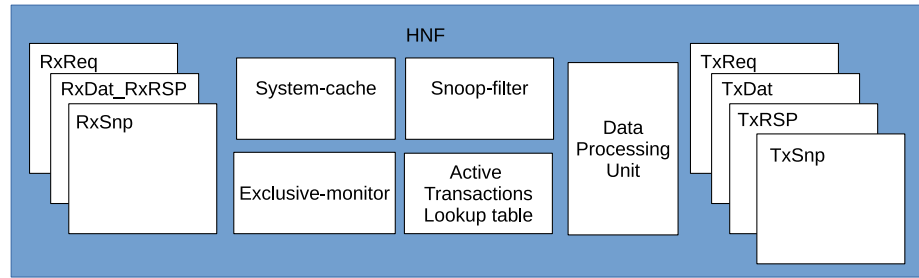


Figure 2: HNF functional block diagram.

- **System-cache:** The system cache is a fully n -ways associative cache which distributed equally between all available home nodes in the system.

The number of ways in system-cache (n) and its depth are parametrizable. The system-cache has two-cycle write and one-cycle read latency.

The system-cache is merged inside the home node. So the communication with system-cache is handled internally inside the home node. However, it is also possible to use the system-cache as an independent CHI-agent (as described in MB2020 specification) by setting `SYS_CACHE_EN` parameter as zero in home node RTL code. In case any data should be updated/captured from the main memory, the home node will forward the corresponding request to its system-cache first. If the data is not available there, the system-cache CHI-agent will forward the request to the main memory.

- **Snoop-filter:** The snoop-filter keeps track of RNFs who have a valid copy of a specific cache-line in a one-hot key format. This allows the home node to indicate the state of the cache line in destination RNFs which is one of the Invalid (no RNF have a valid copy), Unique (only presented in one RNF) or Shared (more than one RNFs have a valid copy) states. The snoop-filter also reduces the sending of unnecessarily snoop requests to RNFs not having a valid copy of a cache line. The snoop-filter also has a single busy bit field that indicates if the corresponding cache line has an active (not yet completed) transaction on the fly. In case, the home node gets a new request to a busy cache-line a Hazard situation is detected. This request is not proceed and send back to the end of input queue.

The snoop-filter is implemented with architecture similar to system-cache. It has limited number of ways which is parametrizable. Once, all ways in an specific cache row are full, the snoop-filter selects one random non-busy field to be evicted. The selected addresses are sent to eviction buffer and wait for eviction transactions to be completed.

Once a request is accepted in a snoop-filter row, the home node does not accept (result in a hazard) another request targeting to the same row for at least two clock cycles. This is necessary for preventing some racing condition in the snoop-filter such as in setting busy bit or selecting an empty way for eviction.

- **Exclusive-monitor** The exclusive monitor keep track of the exclusive requests using two internal modules in parallel: a Minimum single bit monitor and a Partially address monitor. An Exclusive Store transaction is permitted to progress if one of the following occurs:
 1. The address monitor has registered an exclusive sequence for a matching address from the same LP and has not been reset by an Exclusive Store transaction from a different LP with a matching address.
 2. The minimum single-bit monitor has been set by an exclusive sequence from the same LP, and it has not been reset by an Exclusive Store transaction from a different LP to any address.

- **RX/TX CHI-agents:** These modules are developed for handling the communication with Dat, Rsp, Req, and Snp channels. The RX agents include FIFOs and buffer management agents. Once a request is waiting to be serviced inside the input queue its address is sent to cache, snoop filter, and exclusive monitor in one clock cycle in advance, as all of the aforementioned modules have one-cycle read latency.

Once a request is accepted inside the home node, the requested cache block may be provided using one of the following types of transactions:

- Direct memory transaction (DMT): The requested cache block is directly transferred from the main memory to its requester.
 - Direct cache transaction (DCT): The requested cache block is directly transferred from another request node local cache to its requester.
 - System-cache transaction (LCT): The requested cache block is directly transferred from system-cache inside the home node to its requester.
 - Indirect memory transaction (IDMT): The requested cache block is first captured from the main memory and sent to the home node then the home node forwards it to its requester.
 - Indirect cache transaction (IDCT): The requested cache block is first captured from another request node local cache and sent to the home node then the home node forwards it to its requester.
- **Active transaction look-up table:** The home-node requires to keep track of several different types of information for all active transactions (maximum of 256). This includes the waiting list, data field, type of transaction (DCT, DMT ...), request opcode and etc. The waiting list indicates which of the RNFs have pending response to the corresponding active transaction.
 - **Data processing unit (DU)** Dependent on each active transaction request type, the Data processing unit (DU) may be required to execute specific process (i.e arithmetic operation, bypass the initial data or merging initial with dirty data) on up to two 512-bit data (transaction and initial data). The initial data is the one that has equal value with the data saved in the main memory whereas the transaction data is the one which is dirty or comes with Atomic requests. Note that the HNF can receive the transaction and the initial data in any order, (e.g. the initial data first then the transaction or vice versa). The data which is received first is saved in HNF transaction data lookup table (DLKPT). DLKPT is a 512-bit, 256 (maximum active transaction number) entry memory, where the generated data in DU will be always saved there. It is the duty of the DU controlling units to indicate which of the current data or the one which is saved in DLKPT is the initial data.

The initial and txn data are passed to a 64-bit arithmetic logic unit (ALU). The ALU is controlled by `opt` opcode. The ALU support 10 different opcodes. 8 Atomic load/store opcodes as specified in AMBACHI specification (ADD, CLR, EOR, SET, SMAX, SMIN, UMAX, UMIN), one SWAP operation which sends transaction data to the output and one BPASS operation which redirects the initial data to the output.

The size and address fields are used to generate a 64-bit MASK signal which indicates the location where the ALU output should be replaced in the initial data to obtain the final DU data value. According to AMBACHI specification each of the AtomicStore operations apply to 1, 2, 4, or 8 byte data sizes. However, supporting small atomic size (i.e. 1 & 2) results in higher area overhead due to large multiplexing size whereas these sizes may not be used by the processors. In order to reduce the hardware complexity a parameter is defined in the proposed design RTL code (`MIN_SIZ`) which defines the minimum required transaction size. Setting larger `MIN_SIZ` reduces the complexity of converter module and merging mask module (e.g the MASK signal reduces from 64 bit to only 2 bits if the `MIN_SIZ` is set as 32-bit).

4 Transaction flows

In all type of requests that results in data transfer if `likelyshared` and `exclusive` flags are not asserted the coherent NoC may decide to make a DCT or DMT transaction. In this case the transferred data would not be saved in system-cache at the end of transaction. If one or both of these flags are asserted in request flit the NoC is prohibits the DCT/DMT.

4.1 ReadShared Transaction

A ReadShared is a result of read miss in request node local cache. A ReadShared request which is received in a HNF may results in different transaction scenarios:

1. snoop-filter status is invalid and the system-cache misses, the `likelyshared` and `exclusive` flag are de-asserted.

The required data is not shared anywhere in the system, and is not probably shared by other RNs (as `likelyshared==0`) in future. The HNF perform a DMT.

- (a) The home node sends ReadNoSnp request to SNF with filling the `returnnid` and `returntxnid` with original requester node IDs.
- (b) The SNF detect a DMT by observing different `returnnid` and `srcid` fields in request packet and send `ReapData_U` directly to the requester.
- (c) The RN get the data, update it in its cache as unique, sends a `CompAck` response and release its transaction.

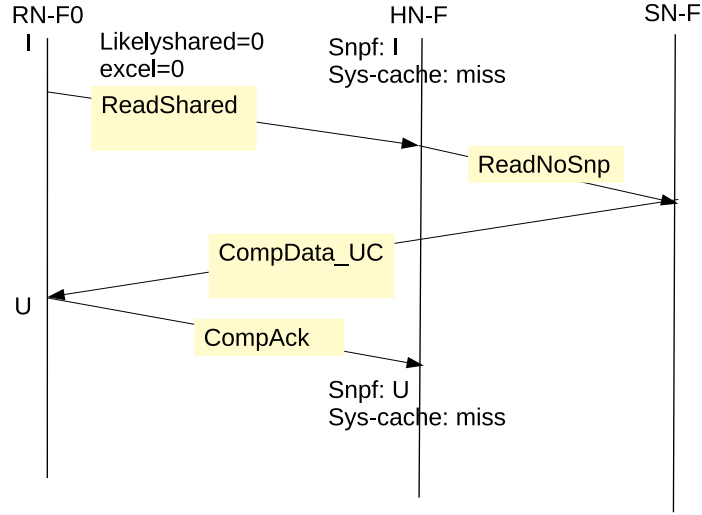


Figure 4: ReadShared.DMT.

- (d) HNF receives CompAck and update the snoopfilter. The busy flag is cleared and the RN-F0 flag is asserted for that specific address.
2. snoop-filter status is invalid and the system-cache misses, the **likelyshared** flag is asserted.

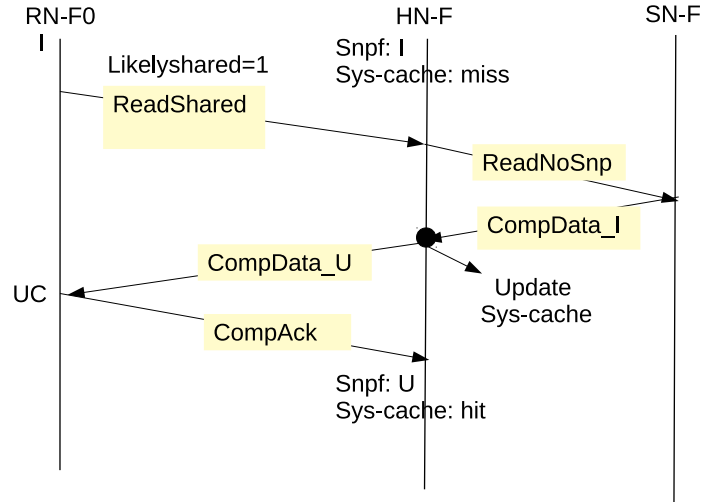


Figure 5: ReadShared.IDMT.

As the **likelyshared** flag is asserted for this transaction, the homenode prefer to save a copy of this data inside system-cache so it sends the

ReadNosnp to SNF with returnid set to home node id. The SNF response detect indirect memory transfer (IDMT) by observing identical **returnid** and **srcid** fields in request packet and sends **ReapData_U** to home node.

- **My Observation:** Optionally once the snoopfilter state is invalid and the excl bit is not asserted the home node may prefer to establish a DMT instead of IDMT, regardless of the value of **likelyshared** flag. As the copy of the data which is saved in syscache cannot be used later unless the requester node silently evict its cache copy. Note that the requester cache copy is in Unique state and RN can modify its value without informing the home node. So home node should always ask it from the requester node later.

3. Snoop-filter status is invalid and the system-cache hit.

The requested cache block is not existed in any RNs but it is valid in system-cache. In this case the data is sent directly from home node to requester node.

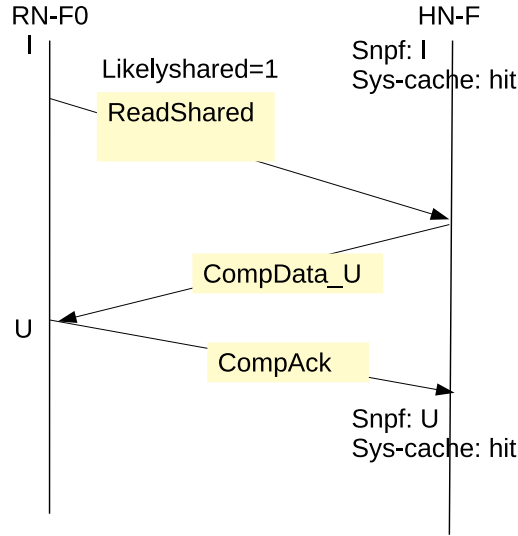


Figure 6: ReadShared,Local cache transfer (LCT).

4. Snoop-filter status is Unique. The data is in Unique ad clean state inside another requester node. The **likelyshared** and **exclusive** flag are de-asserted.

The requested cache block existed in Unique state inside one of the RNs except the one who send the ReadShrared request. We cannot use the sys cache value in case a hit happens as the RN holding the cache copy might changed the cache value without notifying the home node. However,

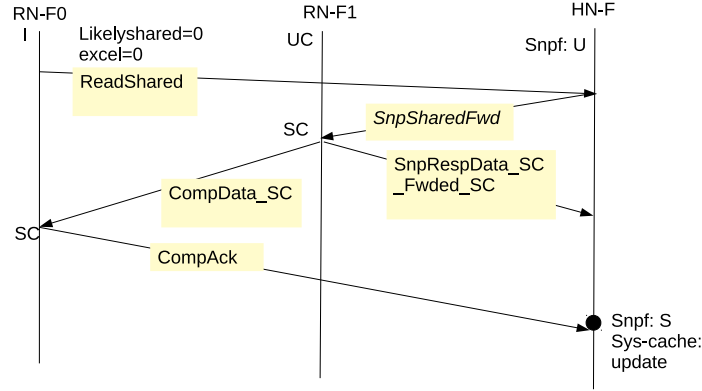


Figure 7: ReadShared, Direct cache transfer (DCT).

the system-cache Data will be saved in transaction LKPT to be sent to requester node in case of DCT failure.

5. Snoop-filter status is Unique. The data is in Invalid state inside all other requester node. The **likelyshared** and **exclusive** flag are de-asserted.

This situation happens due to silent cache transition inside the RN who previously holding the cache copy in Unique state without notifying the home node. In this case the SnpSharedFwd results in SnpResp.I.

Here two conditions can happens:

- (a) The system-cache got a hit at time we have received the ReadShared request so we have saved the system-cache data in LKPT. the home node will remove the SnpResp.I sender from snoop filter and sends the LKPT data with VompData.U reponse to the requester node.
- (b) The sys cache has shown a miss at the request receiving time. In this case we need to obtain the data from main memory.

Once we get the SnpResp.I answer we update the Snoopfilter with removing the RN_F1 from valid copy holders. We also needed to update the waiting list: Remove the waiting flag for both RN_F0 and RN_F1 and assert the waiting flag of SNF in waiting list.

Once the Data has been received from SNF we update the cache state and send it to the requester node.

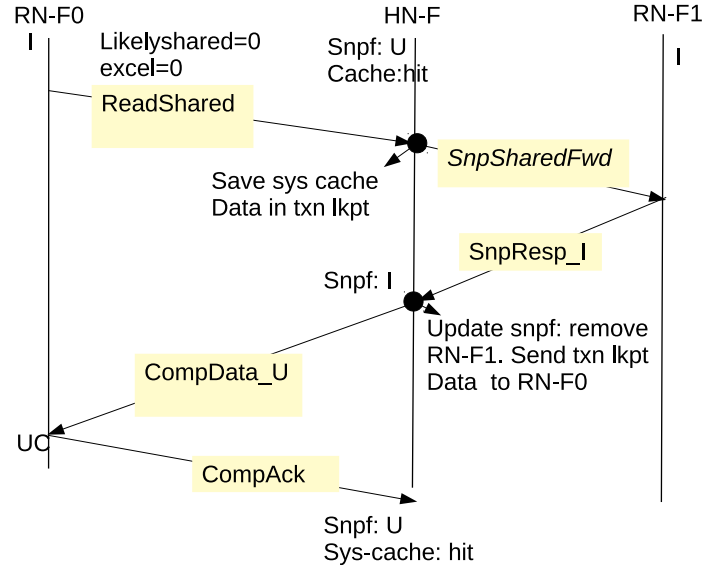


Figure 8: ReadShared, Direct cache transfer (DCT), The syscache hit and DCT failed.

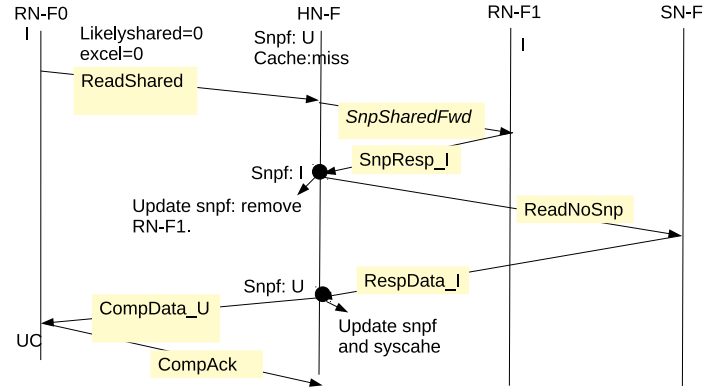


Figure 9: ReadShared, Direct cache transfer (DCT), The syscache miss and DCT failed.

6. Snoop-filter status is Unique. The data is in Unique and Dirty state inside another requester node. The **likelyshared** and **exclusive** flag are de-asserted.

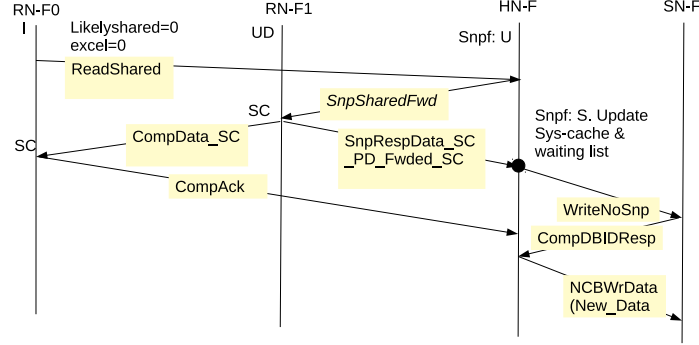


Figure 10: ReadShared,Direct cache transfer (DCT), The cache states is in Unique Dirty.

7. Snoop-filter status is Unique. The **likelyshared** flag is asserted. In this condition the home node sends a *SnpShared* request to snoop channel while the **rettosrc** flag is asserted. Depends on the cache state of RN having the valid copy of the data the following conditions can happens:

- (a) The RN cache state is Unique and Clean. The RN replys to snoop request with *SnpRespData.SC*. The home node update the system-cache and forward the data with *CompData.SC* to the requester node.

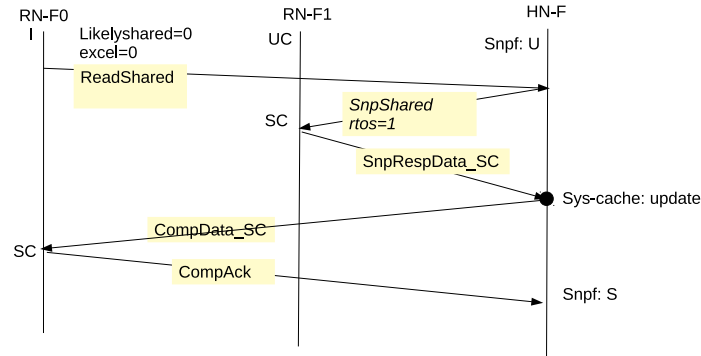


Figure 11: ReadShared,Indirect cache transfer (IDCT), The cache states is in Unique Clean.

- (b) The RN cache state is Unique and Dirty. In this condition the home node receives *SnpRespDat.SC_PD* from the destination RN. The

dirty Data should be written to memory by the home node. So the HNF sends a WriteNoSnp request to main memory and CompData.SC to requester node. As the waiting list is changed the home node update the waiting list with request node and SNF flags are asserted.

The transaction is finished once both Requester node and SNf replied to home node.

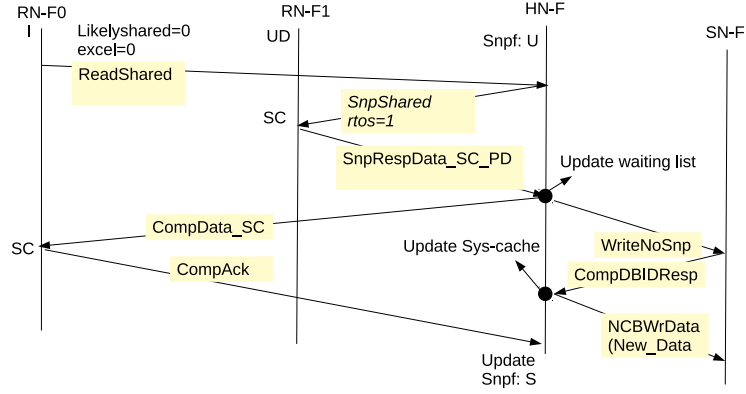


Figure 12: ReadShared, Indirect cache transfer (IDCT), The cache states is in Unique Dirty.

- (c) The holder RN cache state is Invalid due to a silent cache transition and the system-cache hits. The IDCT transaction is similar to DCT transaction in Figure 8.

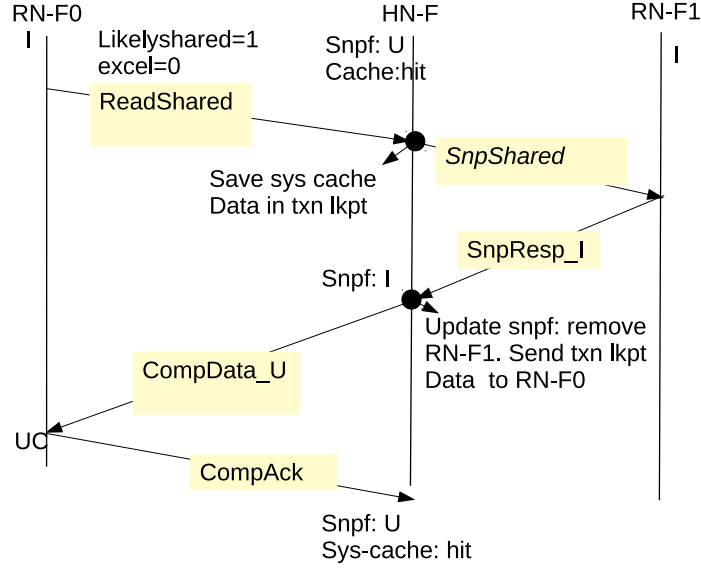


Figure 13: ReadShared, Indirect cache transfer (IDCT), The target RN cache states is Invalid, Syscache hit.

- (d) The holder RN cache state is Invalid due to a silent cache transition and the system-cache miss. The IDCT transaction is similar to DCT transaction in Figure 9.

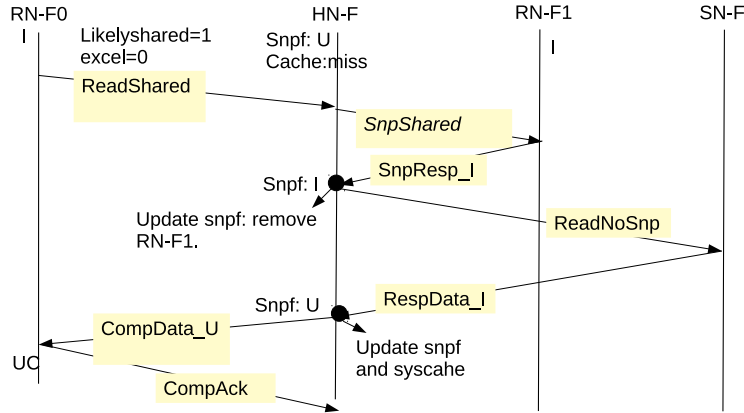


Figure 14: ReadShared, Indirect cache transfer (IDCT), The target RN cache states is Invalid, Syscache miss.

8. The Snoop-filter status is Shared, system-cache hit: Which means more than one RN having a valid copy of the Data. As the requested data value existed in system-cache we can directly send it to the requester using

CompData_SC without the need for sending any snoop requests to the RNs having the valid copies of the data. As the status is in shared in SNPF, the actual status in holder RNs is either Shared or invalid (due to silent cache transition).

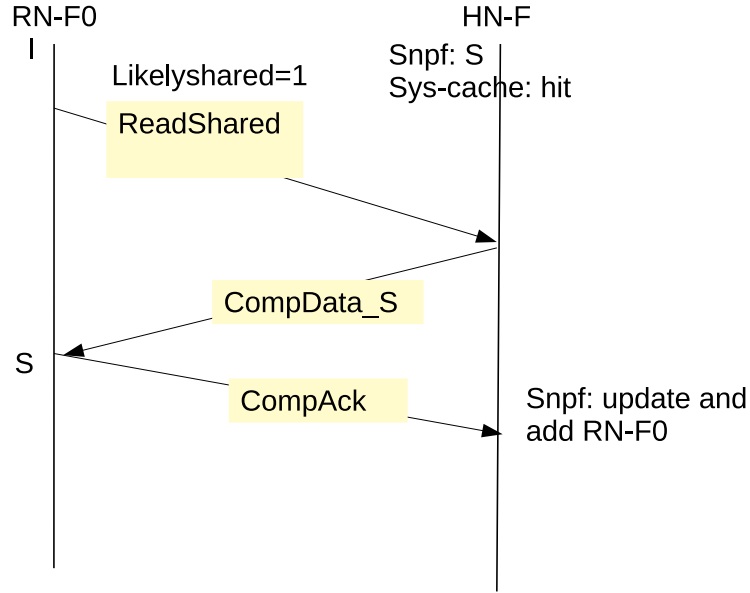


Figure 15: ReadShared, SNPF status is Shared. System-cache hit.

9. The Snooper-filter status is Shared but the System-Cache miss. Here the home node is not permitted to start DCT, as it may cause that the requester RN get data from multiple other RNs (the ones having the valid copies) which is not specified in AMBA CHI standard.

The HNF sends SnpShared to all RNs having the valid copies and wait for receiving data from all RNs. Once all RNs replied to the SnpShared request, if at least one the RN replied by providing data, the sys cache is updated with that data and it will be send to Requester RN.

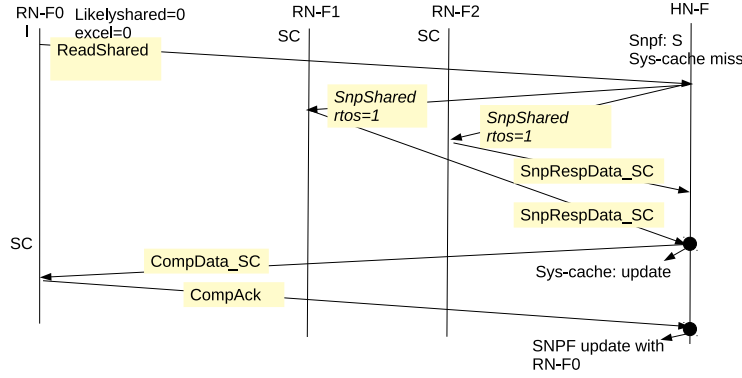


Figure 16: ReadShared, SNPF status is Shared. System-cache miss.

in case none of the RNs replies with Data, the home node has to fetch the data from the main memory. The rest of transaction is similar to Figure 14.

4.2 Atomic Transaction flows

All atomic load store and swap transactions are supported by Coherent ProNoC. The only transaction which is not yet supported is AtomicCompare. Once a home node receives an atomic request, it can lead to three different transaction scenarios dependent on snoop-filter results:

1. **Shared or Unique:** The snoopfilter shows that the valid copy of the data is located in one or (in unique state) or several other request nodes (in shared state).
 - (a) The HNF injects two types of flits, **SnpUnique** snoop requests to all RNs having the valid copy of the data to invalidate their local copies and a **CompDBIDResp** to the RN who initiates the atomic transaction in order to ask him to provide the transaction data. Note that the **SnpUnique** is sent with asserted **rettosrc** flag which causes, the RNs return data to home node before invalidation.
 - (b) HNF waits until receives initial (old) Data from RN(s) having the valid copy of the data and the transaction data from the atomic requester node, then execute the atomic instruction, save the result in cache, send the initial data to requester node and send a **WriteNoSnpf** request to the slave node to get permission to send the results to the main memory later.
 - (c) Once the home node receives **ComDBIDresponse** from SNF, it ends the transaction and send the new data to slave node.

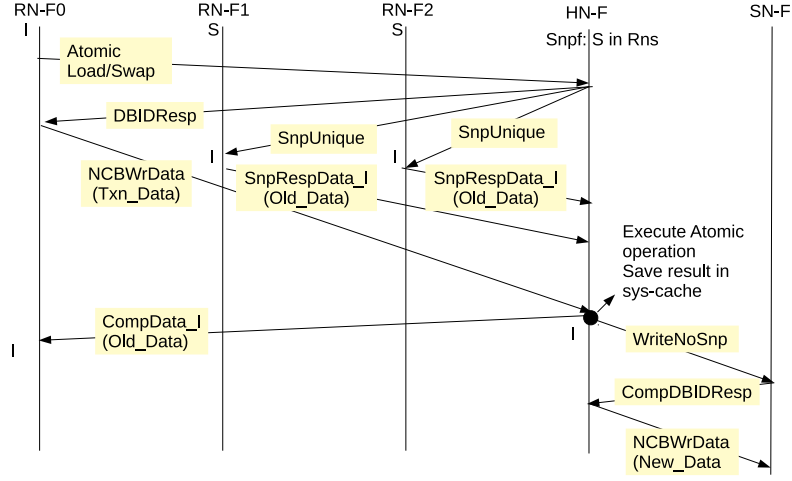


Figure 17: Atomic load/swap transactions, when the snoopfilter is in shared or unique state.

2. **Invalid in RNs but valid in system-cache:** None of the RNs have the valid copy of the data but the valid copy of the data is stored in system-cache.
 - (a) The HNF save the system-cache data (initial data) to transaction lookup table and send a **CompDBIDResp** to atomic requester RN to provide the transaction data.
 - (b) HNF waits until receives the transaction data, then execute the atomic instruction, save the result in cache, send the initial data to requester node and send a **WriteNoSnpf** request to the slave node to get permission to send the results to the main memory later.
 - (c) Once the home node receives **ComDBIDresponse** from SNF, it ends up the transaction and send the new data to the slave node.
3. **Invalid in both SNPF and System-cache:** The initial data is only existed in the main memory.
 - (a) The HNF injects two types of flits, a **WriteNoSnpf** requests to SNF to get write permission and a **CompDBIDResp** to the RN who initiates the atomic transaction in order to ask him to provide the transaction data.
 - (b) HNF waits until receives initial (old) Data from SNF and the transaction data from the atomic requester node, then execute the atomic instruction, save the result in the system-cache, sends the initial data to requester node and sends a **WriteNoSnpf** request to the slave node to get permission to send the results to the main memory later.

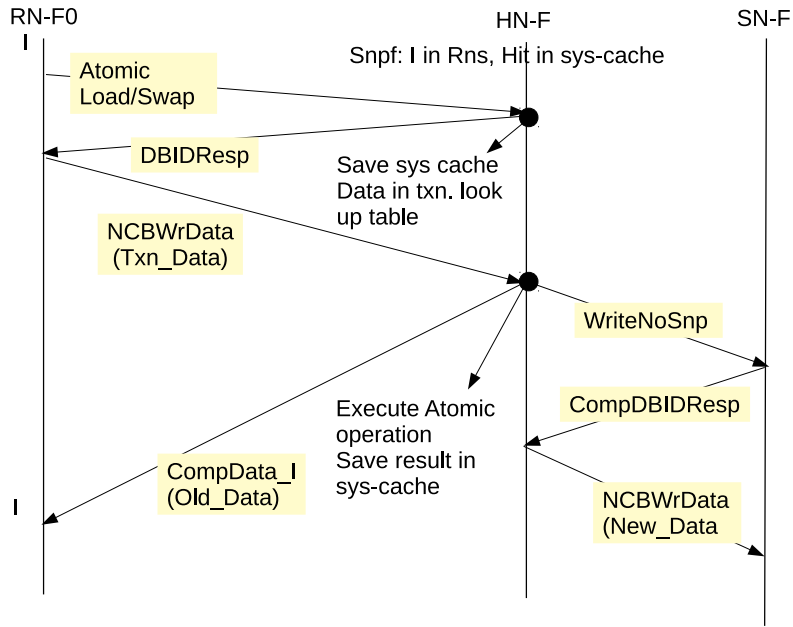


Figure 18: Atomic load/swap transactions, when the snooper shows data is invalid in all RNs but valid in system-cache.

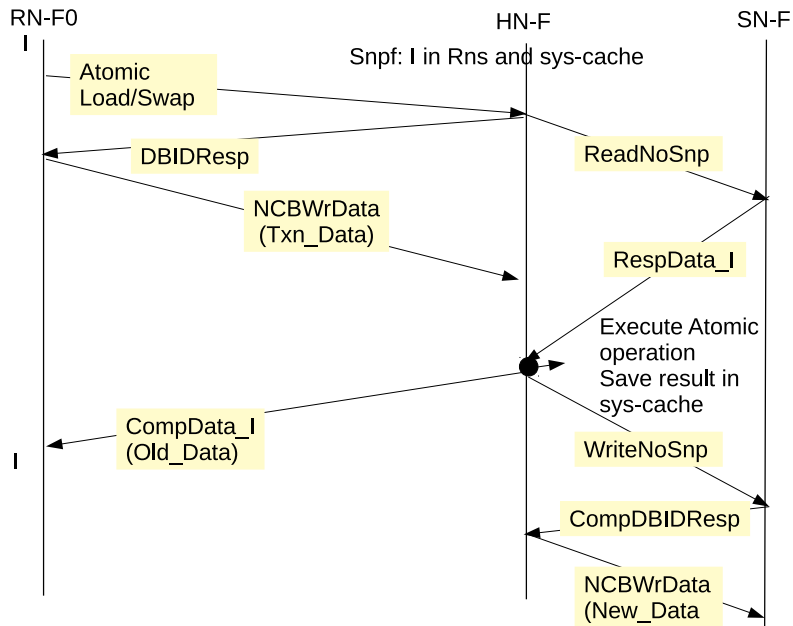


Figure 19: Atomic load/swap transactions, when the snooper shows data is invalid in all RNs and system-cache.

- (c) Once the home node receives `ComDBIDresponse` from SNF, it ends up the transaction and send the new data to the slave node.
- Note: Atomic Store transactions have similar transaction communication pattern with Atomic load/Swap transaction except that instead of sending initial data to the requester node after execution of atomic instruction, the home node sends `Comp_I` instead.

References

- [1] MONEMI, A., TANG, J. W., PALESI, M., AND MARSONO, M. N. Pronoc: A low latency network-on-chip based many-core system-on-chip prototyping platform. *Microprocessors and Microsystems* 54 (2017), 60–74.